

@test_notifier

Appears in: [DL-08](#)

tests/test_notifier.py:1-261

"The whole suite runs in well under a second, and all 22 passed (@test_notifier)."

@test_notifier

tests/test_notifier.py:1-261

```
1  """Tests for server/notifier.py
2
3  Run with:  python -m pytest tests/test_notifier.py -v
4
5  All tests use mocking so no real email or desktop APIs are called.
6  """
7  import sys
8  from pathlib import Path
9  from unittest.mock import MagicMock, patch, call
10
11  import pytest
12
13  # Make sure the project root is on the path
14  sys.path.insert(0, str(Path(__file__).parent.parent))
15
16  # — Helpers —————
17
18  def _reset_config(**overrides):
19      """Reload server.config with custom env-var state."""
20      import importlib
21      import server.config as cfg
22      defaults = dict(
23          NOTIFY_DESKTOP=True,
24          NOTIFY_EMAIL="jenny@company.com",
25          SMTP_HOST="smtp.gmail.com",
26          SMTP_PORT=587,
27          SMTP_USER="betsy@company.com",
28          SMTP_PASS="secret",
29          NOTIFY_ON_APPROVAL=True,
30          NOTIFY_ON_AUTO_PO=True,
31          NOTIFY_ON_SCORE_DROP=True,
32          NOTIFY_ON_DUPLICATE=True,
33          SCORE_DROP_THRESHOLD=0.6,
34      )
35      defaults.update(overrides)
36      for k, v in defaults.items():
37          setattr(cfg, k, v)
38      return cfg
39
40
41  # — Desktop notification tests —————
42
43  class TestDesktopNotification:
44      def test_calls_player_when_desktop_enabled(self):
45          cfg = _reset_config(NOTIFY_DESKTOP=True)
46          mock_player = MagicMock()
47          with patch.dict("sys.modules", {"player": mock_player, "player.notification": mock_player.notification}):
48              import importlib, server.notifier as n
49              importlib.reload(n)
50              n._desktop("Test title", "Test message")
51              mock_player.notification.notify.assert_called_once()
52              kwargs = mock_player.notification.notify.call_args.kwargs
53              assert kwargs["title"] == "Test title"
54              assert kwargs["app_name"] == "Betsy AI"
55
56      def test_noop_when_desktop_disabled(self):
57          cfg = _reset_config(NOTIFY_DESKTOP=False)
58          mock_player = MagicMock()
59          with patch.dict("sys.modules", {"player": mock_player, "player.notification": mock_player.notification}):
60              import importlib, server.notifier as n
61              importlib.reload(n)
62              n._desktop("Title", "Message")
63              mock_player.notification.notify.assert_not_called()
64
65      def test_truncates_message_to_256_chars(self):
66          cfg = _reset_config(NOTIFY_DESKTOP=True)
67          long_msg = "x" * 400
68          mock_player = MagicMock()
69          with patch.dict("sys.modules", {"player": mock_player, "player.notification": mock_player.notification}):
70              import importlib, server.notifier as n
71              importlib.reload(n)
72              n._desktop("T", long_msg)
73              sent = mock_player.notification.notify.call_args.kwargs["message"]
74              assert len(sent) <= 256
75
76      def test_swallows_player_exception(self):
77          cfg = _reset_config(NOTIFY_DESKTOP=True)
78          mock_player = MagicMock()
79          mock_player.notification.notify.side_effect = RuntimeError("display not found")
80          with patch.dict("sys.modules", {"player": mock_player, "player.notification": mock_player.notification}):
81              import importlib, server.notifier as n
82              importlib.reload(n)
83              n._desktop("T", "M") # must not raise
84
85
86  # — Email notification tests —————
87
88  class TestEmailNotification:
89      def test_email_worker_sends_correct_headers(self):
90          _reset_config()
91          import importlib, server.notifier as n
92          importlib.reload(n)
93          with patch("smtplib.SMTP") as mock_smtp_cls:
94              mock_smtp = MagicMock()
95              mock_smtp_cls.return_value.__enter__ = MagicMock(return_value=mock_smtp)
96              mock_smtp_cls.return_value.__exit__ = MagicMock(return_value=False)
97              n._email_worker("Test subject", "<p>hello</p>")
98              mock_smtp.sendmail.assert_called_once()
99              args = mock_smtp.sendmail.call_args
100              assert "betsy@company.com" in args[0]
101              assert "jenny@company.com" in str(args[0])
102
103      def test_email_noop_when_no_email_configured(self):
104          cfg = _reset_config(NOTIFY_EMAIL="", SMTP_USER="")
105          import importlib, server.notifier as n
106          importlib.reload(n)
107          with patch("threading.Thread") as mock_thread:
108              n._email("Subject", "<p>body</p>")
109              mock_thread.assert_not_called()
110
111      def test_email_noop_when_no_smtp_host(self):
112          cfg = _reset_config(SMTP_HOST="")
113          import importlib, server.notifier as n
114          importlib.reload(n)
115          with patch("threading.Thread") as mock_thread:
116              n._email("Subject", "<p>body</p>")
117              mock_thread.assert_not_called()
118
119      def test_email_spawns_daemon_thread(self):
120          _reset_config()
121          import importlib, server.notifier as n
122          importlib.reload(n)
123          with patch("threading.Thread") as mock_thread:
124              n._email("Subject", "<p>body</p>")
125              mock_thread.assert_called_once()
126              assert mock_thread.call_args.kwargs.get("daemon") is True
127
128      def test_email_worker_swallows_smtp_exception(self):
129          _reset_config()
130          import importlib, server.notifier as n
131          importlib.reload(n)
132          with patch("smtplib.SMTP", side_effect=ConnectionRefusedError("no server")):
133              n._email_worker("Subject", "<html></html>") # must not raise
134
135
136  # — Public notification function tests —————
137
138  class TestNotifyApprovalRequired:
139      def test_fires_when_toggle_on(self):
140          _reset_config(NOTIFY_ON_APPROVAL=True)
141          import importlib, server.notifier as n
142          importlib.reload(n)
143          with patch.object(n, "_desktop") as md, patch.object(n, "_email") as me:
144              n.notify_approval_required({
145                  "action": "generate_po", "sku_id": "SKU-003",
146                  "supplier_id": "SUP-01", "po_total": 8597.0,
147                  "reasoning": "Critical stockout risk",
148              })
149              md.assert_called_once()
150              me.assert_called_once()
151
152      def test_noop_when_toggle_off(self):
153          _reset_config(NOTIFY_ON_APPROVAL=False)
154          import importlib, server.notifier as n
155          importlib.reload(n)
156          with patch.object(n, "_desktop") as md, patch.object(n, "_email") as me:
157              n.notify_approval_required({"action": "generate_po", "sku_id": "SKU-003"})
158              md.assert_not_called()
159              me.assert_not_called()
160
161      def test_action_label_mapping(self):
162          _reset_config(NOTIFY_ON_APPROVAL=True)
163          import importlib, server.notifier as n
164          importlib.reload(n)
165          with patch.object(n, "_desktop") as md, patch.object(n, "_email"):
166              n.notify_approval_required({"action": "flag_duplicate", "sku_id": "SKU-004"})
167              title = md.call_args[0][0]
168              assert "SKU-004" in title
169
170
171  class TestNotifyScoreDrop:
172      def test_fires_when_toggle_on(self):
173          _reset_config(NOTIFY_ON_SCORE_DROP=True)
174          import importlib, server.notifier as n
175          importlib.reload(n)
176          with patch.object(n, "_desktop") as md, patch.object(n, "_email") as me:
177              n.notify_score_drop("QuickShip", 0.65, 0.55, "PO-001")
178              md.assert_called_once()
179              me.assert_called_once()
180
181      def test_noop_when_toggle_off(self):
182          _reset_config(NOTIFY_ON_SCORE_DROP=False)
183          import importlib, server.notifier as n
184          importlib.reload(n)
185          with patch.object(n, "_desktop") as md, patch.object(n, "_email"):
186              n.notify_score_drop("QuickShip", 0.65, 0.55, "PO-001")
187              md.assert_not_called()
188
189      def test_supplier_name_in_desktop_message(self):
190          _reset_config(NOTIFY_ON_SCORE_DROP=True)
191          import importlib, server.notifier as n
192          importlib.reload(n)
193          with patch.object(n, "_desktop") as md, patch.object(n, "_email"):
194              n.notify_score_drop("FastParts", 0.70, 0.45, "PO-002")
195              assert "FastParts" in md.call_args[0][0]
196
197
198  class TestNotifyAutoApproved:
199      def test_fires_when_toggle_on(self):
200          _reset_config(NOTIFY_ON_AUTO_PO=True)
201          import importlib, server.notifier as n
202          importlib.reload(n)
203          with patch.object(n, "_desktop") as md, patch.object(n, "_email") as me:
204              n.notify_auto_approved({
205                  "po_id": "PO-001", "sku_id": "SKU-001",
206                  "supplier_id": "SUP-01", "total_amount": 1200.0,
207              })
208              md.assert_called_once()
209              me.assert_called_once()
210
211      def test_noop_when_toggle_off(self):
212          _reset_config(NOTIFY_ON_AUTO_PO=False)
213          import importlib, server.notifier as n
214          importlib.reload(n)
215          with patch.object(n, "_desktop") as md, patch.object(n, "_email"):
216              n.notify_auto_approved({"po_id": "PO-001", "sku_id": "SKU-001"})
217              md.assert_not_called()
218
219
220  class TestNotifyTest:
221      def test_always_fires_desktop_and_email(self):
222          _reset_config()
223          import importlib, server.notifier as n
224          importlib.reload(n)
225          with patch.object(n, "_desktop") as md, patch.object(n, "_email") as me:
226              n.notify_test()
227              md.assert_called_once()
228              me.assert_called_once()
229
230
231  # — Config.update() tests —————
232
233  class TestConfigUpdate:
234      def test_updates_string_attr(self):
235          import server.config as cfg
236          cfg.NOTIFY_EMAIL = ""
237          cfg.update({"NOTIFY_EMAIL": "new@example.com"})
238          assert cfg.NOTIFY_EMAIL == "new@example.com"
239
240      def test_updates_bool_attr(self):
241          import server.config as cfg
242          cfg.NOTIFY_ON_AUTO_PO = False
243          cfg.update({"NOTIFY_ON_AUTO_PO": True})
244          assert cfg.NOTIFY_ON_AUTO_PO is True
245
246      def test_updates_int_attr(self):
247          import server.config as cfg
248          cfg.SMTP_PORT = 587
249          cfg.update({"SMTP_PORT": 465})
250          assert cfg.SMTP_PORT == 465
251
252      def test_ignores_unknown_keys(self):
253          import server.config as cfg
254          original = cfg.NOTIFY_EMAIL
255          cfg.update({"UNKNOWN_KEY": "whatever"})
256          assert cfg.NOTIFY_EMAIL == original # unchanged
257
258
259  if __name__ == "__main__":
260      pytest.main([__file__, "-v"])
```

What it shows: The 22 mocked tests (nothing real sent): desktop on/off/trim/swallow, email headers/silence/thread/swallow, each toggle, and config type-coercion.

Source: tests/test_notifier.py:1-261 · image rendered by [build_snippets.py](#)